Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 3: Implementation of Software Testing – Test Case Generation, Jest, JUnit, Selenium, and Load Testing

Submitted By:

Akisa Poudel

BESE2023

Roll no. 8

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE :

Implementation of Software Testing – Test Case Generation, Jest, JUnit, Selenium, and Load Testing

AIM:

To understand and implement various software testing strategies including unit testing with Jest and JUnit, UI automation testing with Selenium, and load testing with Artillery

THEORY :

Software testing is the process of evaluating an application to identify defects and ensure it meets requirements. In Agile development, testing is integrated throughout every sprint rather than left to the end.

- Test Case Generation: Identifying inputs, expected outputs, and conditions to validate software behavior.
- Jest: A JavaScript testing framework by Meta, widely used for unit and integration testing of Node.js and React applications.
- JUnit: A Java unit testing framework that uses annotations (@Test, @BeforeEach) to define and run test cases.
- Selenium: A browser automation tool that simulates user interactions (clicks, form submissions, navigation) to test web UIs.
- Artillery: A modern load testing tool that sends simulated HTTP traffic to measure application performance and identify bottlenecks under load.

OBJECTIVES:

- Write and run unit tests using Jest for a JavaScript function.
- Write and run unit tests using JUnit for a Java method.
- Automate a browser interaction using Selenium.
- Run a basic load test using Artillery and interpret results.

ALGORITHM:

1. Write a JavaScript function and create a Jest test file.
2. Run Jest and verify test results.
3. Write a Java method and create a JUnit test class.
4. Run JUnit tests using Maven or directly in IDE.
5. Write a Selenium script to open a browser, navigate to a URL, and verify content.
6. Write an Artillery YAML config to simulate concurrent users.
7. Run Artillery and review the summary report.

CODE :

Jest – JavaScript unit test:

```
// math.js
function add(a, b) { return a + b; }
module.exports = { add };


// math.test.js
const { add } = require('./math');
test('adds 2 + 3 to equal 5', () => {
    expect(add(2, 3)).toBe(5);
});
```

JUnit – Java unit test:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;


public class CalculatorTest {
    @Test
    public void testAdd() {
        assertEquals(5, new Calculator().add(2, 3));
    }
}
```

Selenium – Browser automation (Python):

```
from selenium import webdriver
from selenium.webdriver.common.by import By


driver = webdriver.Chrome()
driver.get('https://www.google.com')
assert 'Google' in driver.title
print('Test Passed:', driver.title)
driver.quit()
```

Artillery – Load test config (load_test.yml):

```
config:
  target: 'http://localhost:3000'
  phases:
    - duration: 30
      arrivalRate: 10
scenarios:
  - flow:
      - get:
          url: '/api/users'
```

OUTPUT :

Jest:

```
PASS math.test.js
  ✓ adds 2 + 3 to equal 5
Tests: 1 passed
```

JUnit:

```
Tests run: 1, Failures: 0, Errors: 0
```

Artillery:

```
Requests completed: 300
Response time p95: 120ms
HTTP 200: 300
```

RESULT :

All testing frameworks were successfully implemented. Jest and JUnit validated unit-level logic, Selenium automated browser interactions, and Artillery measured API performance under simulated load.

CONCLUSION :

Testing is a core Agile practice ensuring software quality at every stage. Automating tests enables faster feedback cycles and supports continuous integration pipelines.